Residual Lean Proof Simplifications

After the existing DiophantineConsistency proposals

Research and implementation review for Vladimir Reshetnikov

26 October 2026

Abstract

This report treats the existing proposal corpus as completed work. The strongest remaining change is to replace the residual block-by-block numerical certificate for the current kernel with a verified Horner accumulator. The common-modulus change is already present at the final reviewed commit; the further replacement reduces that maintained module from 426 to 183 physical lines, including the new helper. A second proposal removes three unnecessary state-encoding declarations from an already proposed semantic module by using the constructor-specific decision for an Option value being none. A third proposal derives rank congruences directly from the signed rank polynomial and a saturated low-mask prefix, avoiding natural-packing bounds in the congruence theorem. These contributions have different evidential status: the first has an exact source patch and a successful whole-data Lean kernel replay; the second is a small deletion against an assumed-applied prior migration; the third is a mathematical interface strengthening with no claimed net line saving. Tempting repetitions were explicitly excluded, including a computability dependency reversal that became prior work when a new incoming archive appeared during this review.

1 Scope, baseline, and recommendation

The final reviewed repository state is

`3b51c69d42ff99bc420698532af3fe1d1f9bd356`.

The review began at `7ba7d102822d`. A late recheck found twelve additional commits, including the adoption of the common-modulus residue proof and four additional incoming ZIP archives. The exclusion review was therefore refreshed before this report was finalized. The initial and final states, changed paths, source hashes, and archive inventories are recorded in the accompanying evidence files. The analysis is tied to these snapshots; it is not a claim about future changes to `main`.

The user requested ideas that remain useful after every earlier proposal is assumed applied. This is a stronger condition than finding verbose code in the current checkout. A mechanism can be excluded even when its patch has not been adopted. In particular, an earlier abstraction that already removes a proof family makes a new client of that abstraction insufficient as a new central proposal.

Proposal	Remaining improvement	Recommendation
Verified Horner evaluation	Replace the remaining 59 numerical chunk certificates and their length/append assembly by one checked whole-row computation.	Adopt first, after the focused production-module checks.
Tag-only Option decision	Remove the state equivalence and private encoding/equality instances retained by the earlier computability migration.	Apply the small cleanup after that migration.
Direct signed rank congruence	State the low-bit rank result before proving a natural packing representation or a bound on the high field.	Add as a small arithmetic interface if separating these dependencies is useful.

The first proposal has a measured current-source reduction. The second has a small measured reduction against a particular prior proposal. The third offers a weaker hypothesis set and a more direct proof route. The report does not add those benefits into a single line-saving total, and it does not manufacture further proposals from variations of the same mechanism.

1.1 What the review excludes

The earlier corpus is extensive: 43 placed proposal packets, their adoption record, the intake index, and seven incoming ZIP archives available at the final pin, for 50 packet packages. The incoming archives contain 2,051 member entries; the combined text inventories contain 4,155 entries, including supporting code and copied sources. The principal-report audit initially indexed 117 TeX inputs; supporting Lean files, patches, Markdown notes, and archived copies were also searched for the candidate mechanisms. The four late-arriving archives were inspected separately and added to the final audit. A file inventory and a search are not the same as reading every copied production file or build log word for word. The evidence distinguishes those activities.

The retained ideas are repository-relative findings. There is no claim that Horner evaluation or modular reduction is new mathematics. The relevant novelty is that these particular proof transformations are not already proposed for these remaining obligations in the supplied exclusion corpus.

2 Proposal I: eliminate the remaining chunk certificate with Horner evaluation

2.1 The precise remaining problem

The current `CurrentKernelResidues2011.lean` computes the same fixed 941 selector rows modulo a common modulus

$$M = 47 \cdot 2011 \cdot 120199 = 11\,360\,848\,883.$$

The previous proposal, packet `2026-10-26zn`, already replaced three separate modular computations by one computation followed by homomorphic projections. That is prior work and is not proposed again here. The final reviewed source is byte-identical to the one-modulus implementation in that packet. Its source hash is included in the evidence.[1, 2]

The remaining evaluator has the recurrence

$$E_r([]) = 0, \qquad E_r(a :: \ell) = 2\,a_2\,r^{|\ell|} + E_r(\ell),$$

where the first component of each row records its index and the second component supplies its numerical value. Its interpretation as a descending-power polynomial is

$$E_r([a_0, \ldots, a_{n-1}]) = \sum_{j=0}^{n-1} 2(a_j)_2 \, r^{n-1-j}.$$

The source calls its finite groups Horner blocks, but this recurrence explicitly evaluates a power and a tail length at each head. The final one-modulus proof still checks 59 chunks, proves 58 chunk-length facts, and names their append assembly. The new opportunity is to change the computational certificate route while retaining this evaluator and its mathematical meaning.

2.2 A small verified accumulator

Define

$$H_r([], z) = z, \qquad H_r(a :: \ell, z) = H_r(\ell, zr + 2a_2).$$

The state is the accumulated value of the prefix already consumed. Each update multiplies the accumulator by the base and adds the doubled row coefficient. In particular, it does not ask for the remaining length or compute a separate power of the base at each row.

Proposition 2.1 (Accumulator invariant)**.** *For a commutative semiring, every row list ℓ, and every initial accumulator z,*

$$H_r(\ell, z) = zr^{|\ell|} + E_r(\ell).$$

Consequently $E_r(\ell) = H_r(\ell, 0)$.

Proof. Induct on ℓ, leaving z arbitrary. For the empty list, the right side is $z \cdot 1 + 0$. For $a :: \ell$, apply the induction hypothesis to $zr + 2a_2$:

$$\begin{aligned} H_r(a :: \ell, z) &= (zr + 2a_2)r^{|\ell|} + E_r(\ell) \\ &= zr^{|\ell|+1} + 2a_2 r^{|\ell|} + E_r(\ell) \\ &= zr^{|a::\ell|} + E_r(a :: \ell). \end{aligned}$$

The special case $z = 0$ follows by the zero laws. No numerical property of the row data or the modulus is used. □

The checked implementation is short enough to audit directly:

```
def run {R : Type*} [Semiring R] (r : R) :
    List (ℕ × ℕ) → R → R
  | [], z => z
  | a :: l, z => run r l (z * r + 2 * (a.2 : R))

theorem run_eq {R : Type*} [CommSemiring R] (r : R)
    (l : List (ℕ × ℕ)) (z : R) :
    run r l z = z * r ^ l.length + evaluate r l := by
  induction l generalizing z with
  | nil => simp [run, evaluate]
  | cons a l ih =>
    rw [run, ih, evaluate, List.length_cons, pow_succ]
    ring

theorem evaluate_eq_run {R : Type*} [CommSemiring R]
    (r : R) (l : List (ℕ × ℕ)) :
    evaluate r l = run r l 0 := by
  simp only [run_eq, zero_mul, zero_add]
```

The helper does not alter the public evaluator. It supplies a proved alternative for checking its value. This matters because the evaluator's append law and the physical-order interpretation remain useful interfaces even when the new numerical proof no longer uses the old chunk assembly.

2.3 The actual finite certificate

The standalone replay uses the actual ordered 941 pairs from the repository fixture, with

$$r = 10\,653\,241\,607 \quad \text{in } \mathbb{Z}/M\mathbb{Z}.$$

The resulting residue is

$$E_r(\text{rows}) = 10\,463\,387\,658.$$

After rewriting by `evaluate_eq_run`, one `decide +kernel` command checks this whole list. This command succeeded with the repository's pinned Lean 4.32.0 and mathlib revision. The successful replay is what permits removal of the remaining chunk certificates; a merely plausible claim that the whole list should reduce would not be sufficient.

The order of the rows is essential. Since the exponent attached to a row depends on its position, equality of the row sets would not validate the certificate. The accompanying Python audit checks the complete ordered list and the concatenation of its 59 literal chunks against the maintained physical-data source. The canonical compact JSON representation of the ordered pair list has SHA-256

`362de053598c3bfa3facee341a6b2fc46a66eda74ecf98904a33c7dd3c9209a8`.

The Python calculation is an independent data check. It is not a premise used by the Lean theorem.[3]

2.4 Exact source accounting

The comparison charges the new evaluator and its proof to the proposal:

Measure	Current baseline	Proposed module
Physical source lines	426	183
UTF-8 source bytes	18,230	7,465
Numerical chunk residue theorems	59	0
Chunk-length theorems used in assembly	58	0
Whole-row residue certificate	1 assembled	1 direct

Thus the net source reduction is **243 lines and 10,765 bytes**. These are exact counts for this one current-source comparison, with its new helper included. They are not an exact line-saving total against an imagined tree containing every earlier unmerged proposal: older generic block-assembly work may already remove some append/length boilerplate. The defensible new obligation removed after granting that work is the remaining set of 59 separate numerical chunk certificates. No all-prior-adjusted aggregate total or compiler-performance gain is claimed.

The public `evaluate_append` theorem is retained. The three modulus projections, protected Start contribution, control contribution, physical-order data, reindexing bridge, and interpretation through `rawWord_cast` retain their existing meanings. The retained projection machinery is the already adopted common-modulus work. The new patch only removes the computational decomposition that is no longer needed.

2.5 Why this remains new after the preceding proposals

Packet `zn` changes *where* the calculation is performed: one common modulus. The present change changes *how* the remaining calculation is evaluated: a proved accumulator with one whole-list kernel certificate. Earlier scanner-fold proposals concern proof scanners and their semantic invariants, rather than this fixed numerical row certificate. Those folds can be assumed available without eliminating this remaining certificate family. In addition, the fresh Shape/Windows/Growth packet explicitly discusses and rejects an affine-monoid Horner scheme for chunk assembly as old repackaging. That note is part of the exclusion baseline. The present proposal proves a different executable traversal and checks the entire row list, so it deletes the chunk certificates themselves instead of merely recombining their existing facts.

The relevant comparison is therefore 59 chunk certificates to one whole-row check. Comparing the proposal directly to the old three-modulus source and claiming removal of 177 chunk certificates would count the previous proposal again. The final head update makes the distinction especially concrete: the 426-line file is now maintained source, not merely a hypothetical baseline.

2.6 Adoption plan and practical limits

The archive contains both a complete proposed module and a unified patch against the current one-modulus baseline. Before applying the patch, compare the target's hash with the recorded baseline or inspect any intervening changes. The patch should be rebased normally if the file has moved forward.

The new invariant and the literal whole-list certificate have been kernel checked. The complete production module and its downstream dependency graph have not been rebuilt in this review. Accordingly, the proposed module is a concrete, reviewable implementation whose novel proof mechanism is checked; it is not reported as a completed repository migration.

The focused adoption gate is to compile the changed module, its existing residue checks, and the direct controller-unit client. Compare the old public theorem types and axiom reports. The proof must continue through the maintained physical-row bridge; copying a numerical answer into a disconnected test theorem would not replace that bridge.

No wall-clock or memory speedup is asserted. The recurrence removes repeated powers and list-length traversal from the computational inner loop, but Lean's elaboration, reduction, and kernel costs must be measured before making a performance claim. Successful replay and exact source reduction are the results actually established here.

3 Proposal II: remove the state-encoding scaffold from a tag-only test

3.1 Start after the earlier dependency reversal

The new incoming *ProveIt Proof Simplification Review* already proposes deriving primitive recursiveness from the supplied scalar realizer. Its reference migration extracts an `EnumeratorSemantics.lean` module and removes the parallel primitive-recursive support route. That whole dependency reversal is assumed applied here and is explicitly excluded from the new contributions.[8]

A smaller residual remains in the reference semantic module. The prior report says that it conservatively retains `stateEquiv`, a private `Primcodable St` instance, and a private `DecidableEq`

`St` instance. The retained scaffold encodes all six state coordinates. Its purpose in the semantic module is only to support deciding whether a bounded run is equal to `none`.

That observation depends solely on the Option constructor. A result of the form `some s` differs from `none` independently of the value of `s`. There is therefore no need to decide equality between two states or to encode a state as a natural number.

3.2 Use the predicate-specific decision procedure

Lean supplies `Option.decidableEqNone`. The following type can be inhabited with no assumption on the element type:

```
example {α : Type} (o : Option α) : Decidable (o = none) :=
  inferInstance
```

This is narrower than requiring `DecidableEq α` and then building a decision procedure for arbitrary equalities in `Option α`. In the actual semantic module, the relevant definition remains

```
def haltingTest (t : ℕ) : ℕ :=
  Encodable.encode (decide (boundedRun t initialState = none))
```

The encoded value here is a Boolean. The encoding of `St` does not contribute to the result.

The proposed patch removes the state equivalence and the two private instances from the already migrated semantic module. It replaces the broad `Mathlib.Computability.Primrec.Basic` import with `Mathlib.Logic.Encodable.Basic`. The latter retains the infrastructure needed for the Boolean encoding, natural notation, sets, and iteration. Dropping all Mathlib support would not preserve the module's requirements; the patch includes this narrower replacement.

3.3 Concrete benefit and compatibility check

The exact comparison is the incoming migration's **85-line module versus a 76-line module**: nine fewer physical lines and three fewer support declarations, with the replacement import counted. There is no new production helper. This is not a second claim for the removal of 32 private `pr_*` theorems, and it is not a 206-line saving against the old maintained file. Those changes belong to the previous proposal.

The proof obligation is correspondingly precise: compile the cleaned semantic module, show that the original observation and the tag-specific observation are definitionally equal, and preserve the supplied scalar program's specification. The accompanying checks compare the concrete `haltingTest` expression by `rfl`; the validation summary records the exact integration scope actually completed.

The same principle can be useful in other places that test a sum or option constructor, but such additional clients have not been counted here. The recommendation is the concrete three-declaration deletion above. It is small, has no mathematical downside, and removes an unnecessary coupling between the semantic halting observation and the representation of machine states.

4 Proposal III: derive low-bit rank congruences before natural packing

4.1 The dependency that can be removed

The signed rank polynomial is already exposed by `NativeRankVariation.windowRank`:

$$W(q, C, F, M_C, M_F, J) = \big(q(q-F)-C\big)(q^2-1) + (M_C + qM_F)J.$$

It is defined over a commutative ring. The parity proof in `CurrentPackingParity2011` proves that a natural four-block value is congruent to 3 modulo 4, then identifies the constructed rank with that natural value using `fourBlocks_cast`. That identification requires the source positivity and packing range, including $C + F < q$.[4, 5]

Those conditions are useful elsewhere, particularly for bounds and population counts. They are stronger than needed for the congruence itself. The proposed interface proves the residue directly in the signed polynomial, before introducing a natural packed value. This is a simplification of the theorem's dependencies and hypotheses, not a new claim that every surrounding packing bound can be removed.

4.2 The polynomial projection

Lemma 4.1 (Rank projection at a zero base)**.** *Let $\varphi : R \to S$ be a unital ring homomorphism with $\varphi(q) = 0$. Then*

$$\varphi\big(W(q, C, F, M_C, M_F, J)\big) = \varphi(C) + \varphi(M_C)\varphi(J).$$

Proof. Apply the homomorphism to the displayed polynomial. All terms containing q vanish, and

$$(0 - C)(0 - 1) + M_C J = C + M_C J.$$

No sign or range condition on F, M_F, or the rank is involved. □

This elementary identity explains the opportunity. The upper fields are multiplied by the base, so a modulus dividing the base removes them immediately. Reconstructing all the natural packing blocks before making that observation introduces proof obligations that the residue calculation cannot use.

4.3 A saturated low prefix supplies the remaining residue

For $k \in \mathbb{N}$, put $m = 2^k$. Take natural q, C, M_C, J, let $B = M_C J + 1$, and suppose

$$m \mid q, \qquad C > 0, \qquad (C-1) \,\&\, B = 0, \qquad B \bmod m = m - 1.$$

Here & is natural bitwise AND. The last equation says that the lowest k bits of B are all one.

Proposition 4.2 (Direct signed rank residue)**.** *Under these hypotheses, for arbitrary integer F and M_F,*

$$W(q, C, F, M_C, M_F, J) \equiv -1 \pmod{2^k}.$$

Proof. Reduce the mask equation modulo 2^k. The bitwise-modulus law gives

$$((C-1) \bmod m) \,\&\, (B \bmod m) = 0.$$

Since $B \bmod m = m - 1$, and AND with $2^k - 1$ extracts the lowest k bits, this implies

$$(C - 1) \bmod m = 0.$$

The polynomial projection gives $W \equiv C + M_C J$. Because $C > 0$, natural subtraction satisfies $C = (C - 1) + 1$. Therefore

$$C + M_C J = (C - 1) + (M_C J + 1) = (C - 1) + B \equiv 0 + (m - 1).$$

This is -1 in $\mathbb{Z}/m\mathbb{Z}$. □

The positivity condition $C > 0$ must remain when $C - 1$ is natural subtraction. If $C = 0$, then the truncated predecessor is zero regardless of its intended integer value, and the last equality in the proof is unavailable. The case $k = 0$ has modulus one and is harmless.

4.4 The new boundary versus the already known bit argument

The existing `fourBlocks_mod_four` theorem already contains the bitwise prefix argument for $k = 2$. This report does not advertise that argument as new. The additional step is to apply it to the *signed rank polynomial* before any natural four-block representation, and to expose the modulus 2^k as a parameter.

The distinction from prior reflected-parity work also matters. The earlier rejected reflected-parity proposal concerned normalized carry digits. Here the object is the rank polynomial, and the simplification is the removal of the guarded natural-packing detour. The fresh incoming rank proposals address such matters as rank growth, exact upper deficit, zero tests, and rephasing; the accompanying audit records the candidate comparison against those additions.

4.5 A useful separating example

Choose

$$q = 4, \quad C = 1, \quad M_C = 2, \quad J = 1, \quad F = 100, \quad M_F = -7, \quad k = 2.$$

Then $B = 3$, the mask condition is $0 \,\&\, 3 = 0$, and $4 \mid q$. The rank is

$$W = (4(4 - 100) - 1)(16 - 1) + (2 - 28) = -5801 \equiv 3 \pmod 4.$$

Thus the new congruence remains meaningful even when $C + F < q$ fails and the signed rank is negative. This example does not describe an admissible positive source solution; its purpose is to separate the algebraic residue statement from the stronger packing statement. It demonstrates why those stronger hypotheses are unnecessary for this theorem.

4.6 How the current consumer would change

In `CurrentPackingParity2011.packing_rank`, obtain the rank/population witness as before. For the residue subgoal, apply the new direct rank lemma to the source value, the existing low-mask fact, and the existing base-divisibility fact. Transport the resulting integer congruence through the witness equality $(R : \mathbb{Z}) = W$, and conclude $R \bmod 4 = 3$.

The positivity, balance, and $C + F < q$ proof may still be required later in that theorem to establish its lower bound $2(\text{width}\, N) < R$. The proposal removes them from the residue dependency, not automatically from the entire theorem. The existing natural `fourBlocks_mod_four` API should be retained when other clients need the packed representation.

This is the smaller recommendation. A generic helper may cost more lines than it saves in the first client. Its justification is a cleaner theorem boundary, applicability to signed intermediate expressions, and reuse for other low-prefix lengths. There is no measured positive net line reduction for this proposal. It should be adopted for those interface benefits, rather than to improve a line-count metric.

5 Verification, provenance, and trust boundaries

5.1 Pinned environment

The checked pilots use Lean 4.32.0, compiler revision

`8c9756b28d64dab099da31a4c09229a9e6a2ef35`,

and the mathlib revision from the repository manifest,

`81a5d257c8e410db227a6665ed08f64fea08e997`.

The exact commands, source hashes, observed exit codes, and axiom reports are included in the validation files. These versions are provenance for the replay, not a claim about what the latest Lean release should be.[9, 10]

5.2 Completed Lean checks

- **Horner evaluator.** The generic accumulator invariant and exact 941-row numerical certificate compile successfully. The recorded axioms are `propext` for `run_eq`, and `propext, Quot.sound` for `rows_common`. The numerical command is `decide +kernel`, not `native_decide`.
- **Option cleanup.** The complete cleaned 76-line semantic module compiles. The check re-elaborates the original 85-line incoming source in a separate namespace and proves `baseline_eq_cleaned` by `rfl`. This compares the actual old and new `haltingTest` definitions, including instance selection. The actual acceptance and calibration endpoints were audited. Fourteen unchanged semantic dependency modules were freshly compiled from sources whose hashes match the reviewed repository. The exact prior-baseline patch also passes its apply check.
- **Rank prefix.** The signed residue theorem, its natural-number adapter, and both closed stress examples compile successfully. Their reported axioms are `propext, Classical.choice, Quot.sound`.

The generic Option constructor observation is axiom-free. The concrete Option comparison and semantic endpoints use the ordinary axioms `propext, Classical.choice, Quot.sound`, inherited through the actual definitions. None of the retained checked proofs introduces a new axiom or reports `sorryAx`. A proof by `rfl` is a definitional compatibility check; it is not used to claim that its referenced definitions have no axioms.

The checks do not include a full repository build. In particular, the complete Horner production module and all downstream consumers have not been rebuilt, and the rank pilot has not been integrated into its production consumer. The Option target itself was compiled, but the entire already-proposed computability migration and its downstream tree were not replayed. These scopes are preserved in the archive's validation records.

5.3 What is and is not established

The Horner replay proves the general evaluator equality and the actual finite residue. An independent data audit checks ordered-list agreement with the reviewed source; recorded hashes identify the exact files. The retained production code supplies the physical tile interpretation. These are separate checks, and the report does not conflate them.

For the rank proposal, the mathematical point is a weaker hypothesis set for a signed congruence. Its checked prototype and its production integration status are reported separately. None of the proposals asserts consistency of ZFC, changes the logical equivalence being formalized, or claims a new Diophantine equation. The requested work is proof simplification.

The final source snapshot differs from the initial snapshot in five maintained Lean modules, the adoption/review records, and the incoming archive additions. The Horner baseline was re-read from the final state and compared with the earlier assumed-applied source. The final novelty judgment uses the refreshed corpus. This update also invalidated one initially retained proposal; the next section records that exclusion explicitly.

6 Novelty decisions and withdrawn alternatives

A novelty audit is useful only if it can reject attractive candidates. The following mechanisms were considered and excluded from the new recommendations:

Candidate	Reason for exclusion
Derive computability from the supplied scalar program	The late incoming *ProveIt Proof Simplification Review* explicitly proposes consuming the realizer, splitting `EnumeratorSemantics`, and removing the parallel private `pr_*` proofs. This was a promising independent discovery early in the review, but became exact prior work at the final pin. It was withdrawn.
Common modulus for the kernel residues	Packet `zn` supplies the mechanism and patch; it is now merged. Only the residual evaluator/whole-list certificate is proposed here.
Shared scalar/graph representation algebra	Packet `e`, section 14, already shares control flow while retaining separate realizers and costs.
One nonmaximum digit gives a strict Mersenne bound	The maintained `Facts.word_lt` in `CurrentCarryConstruction2011` already proves the criterion. Earlier all-maximum/carry interfaces also cover it.
One nonzero digit gives positivity	Existing word positivity and packet `zj` already supply this mechanism; another client is not a new central idea.
Split/Gap exclusion by difference sets	Packet `p` already proposes projected support differences and translation cancellation, including additional clients.
Generic Option iteration or failure fusion	Earlier scanner/iterator proposals already cover the relevant absorbing-state and semiconjugacy arguments.
Joint evaluation, degree, and cost invariants	Earlier compiler proposals already supply topological invariants and metadata folds.
Sidon moment polarization	Packet `y` contains the exact polarization identity and checked patch. Packet `zm` explicitly assumes it before a further cleanup.

The general code-derived computability prototype was successfully checked during this review, but it is not packaged as a new proposal. Only the separately checked deletion of the state scaffold from that earlier migration is retained. Checking an existing idea can be valuable verification work; it does not meet the user's request for a new recommendation after that idea is assumed applied. Similarly, the report does not relabel the already known low-mask bit argument as a third new mechanism.

7 Recommended sequence

First review the Horner patch. Its target is narrow, its source reduction is exact, and its new computation is already demonstrated by a complete standalone kernel replay. Confirm the baseline hash and run the focused production checks before adopting it.

Apply the nine-line Option-scaffold cleanup only after the earlier computability migration; its patch deliberately targets that migration, rather than reproducing it. Then decide whether rank congruences should be available independently of natural packing bounds. If that separation is useful, add the signed-prefix theorem in a small arithmetic module and use it for the parity subgoal. Preserve the natural packing API for population and magnitude proofs. Measure the complete helper-plus-client change if source reduction is a goal; no such gain is assumed here.

All three proposals are deliberately incremental. They retain the existing mathematical endpoints and make different kinds of remaining proof burden smaller: a concrete certificate family, redundant typeclass infrastructure, and the hypothesis/dependency burden of an arithmetic lemma.

A Archive contents and reproduction

The ZIP contains this compiled report and its complete LaTeX source, the checked Lean pilots, the current-source Horner patch and complete proposed module, validation records, ordered-data checks, novelty inventories, and reproduction instructions. It excludes compiler binaries, cached object files, unrelated repository content, and unsuccessful duplicate candidates.

The Lean pilots can be replayed from a checkout using the repository-pinned toolchain. The README gives the exact commands and distinguishes standalone checks from production integration. The Python verifier accepts a repository path and checks the current ordered physical data against the certificate fixture. The PDF can be rebuilt with XeLaTeX using the supplied build script or Makefile. No generated proof object is required to reconstruct the checks.

The accompanying manifests record file hashes rather than claiming that a search proves absolute novelty. They make the particular review reproducible and let a later maintainer identify exactly which source and excluded proposals were considered.

References

[1] ProveIt, *CurrentKernelResidues2011.lean*, final reviewed commit.
Pinned current one-modulus evaluator, chunk certificates, and residue bridges.

[2] ProveIt, *Lean proof simplification packet 2026-10-26zn*.
Prior common-modulus proposal and complete proposed module.

[3] ProveIt, *WangTiles942PhysicalData.lean*.
Pinned ordered physical row data.

[4] ProveIt, *NativeRankVariation.lean*.
Signed rank polynomial and exact perturbation identities.

[5] ProveIt, *CurrentPackingParity2011.lean*.
Existing natural four-block parity proof and packing consumer.

[6] ProveIt, *Lean proof simplification research corpus and adoption record*.
Pinned exclusion corpus.

[7] ProveIt, *Incoming proposal archives*.
Pinned intake directory including the four late additions.

[8] ProveIt incoming, *ProveIt Proof Simplification Review*, proposal A and reference semantic module.
Pinned prior migration, explicitly retaining the state scaffold.

[9] ProveIt, *lean-toolchain*.
Pinned Lean version.

[10] ProveIt, *lake-manifest.json*.
Pinned dependency revisions.